Week 5: JavaScript: Loops, Arrays, and Functions

This document provides detailed notes on **Looping Statements**, **Arrays**, and **Functions** in JavaScript, including definitions, types, methods, and examples. Each section is organized with serial numbers, and code examples are numbered for clarity. All text is intended to be left-aligned for consistency.

---

1. Looping Statements in JavaScript

1.1 What is a Loop?

A loop is a programming construct that executes a block of code repeatedly based on a condition or a set number of iterations. Loops automate repetitive tasks, reducing redundant code.

1.2 Real-Life Example

Imagine packing 10 boxes for shipping. Instead of manually packing each box, you set up a system to pack them automatically until all 10 are done. In programming, a loop performs this repetitive task until the condition (e.g., 10 boxes) is met.

1.3 Types of Loops

JavaScript supports three main types of loops: `for`, `while`, and `do-while`.

1.3.1 For Loop

- **Description**: Used when the number of iterations is known. It includes initialization, condition, and increment/decrement.

Syntax:

```
 for (initialization; condition; increment/decrement) {
    // Code to execute
}
```

Example 1:

```
 for (let i = 1; i <= 5; i++) {
    console.log(i);
}
// Outputs: 1, 2, 3, 4, 5
```

1.3.2 While Loop

- **Description**: Runs as long as a specified condition is true. Used when the number of iterations is unknown.

Syntax:

```
 while (condition) {
    // Code to execute
}
```

Example 2:

```
 let i = 1;
while (i <= 5) {
    console.log(i);
    i++;
}
// Outputs: 1, 2, 3, 4, 5
```

1.3.3 Do-While Loop

- **Description**: Similar to the `while` loop but guarantees at least one execution before checking the condition.

Syntax:

```
 do {
    // Code to execute
} while (condition);
```

Example 3:

```
 let i = 1;
do {
    console.log(i);
    i++;
} while (i <= 5);
// Outputs: 1, 2, 3, 4, 5
```

---

2. Arrays in JavaScript

2.1 What is an Array?

An array is a data structure that stores multiple values in a single variable, accessible via zero-based indices. Arrays are objects in JavaScript and inherit properties from `Array.prototype`.

Example 4:

```
 let fruits = ["Apple", "Banana", "Orange"];
console.log(fruits[0]);
// Outputs: Apple
```

2.2 Array Length

- **Description**: The `length` property returns the number of elements in an array. It can also truncate or extend an array.

Example 5:

```
 let cities = ["New York", "London", "Tokyo"];
console.log(cities.length); // Outputs: 3
cities.length = 2; // Truncates to ["New York", "London"]
console.log(cities);
// Outputs: ["New York", "London"]
```

2.3 Array Methods

Array methods are built-in functions for manipulating arrays, categorized as mutator, accessor, iteration, and other methods.

2.3.1 Mutator Methods

These methods modify the original array.

1. **push() :-** Adds elements to the end of an array and returns the new length.

Example 6:

```
let numbers = [1, 2, 3];
numbers.push(4, 5);
console.log(numbers); // Outputs: [1, 2, 3, 4, 5]
console.log(numbers.push(6)); // Outputs: 6
```

2. **pop():-** Removes and returns the last element.

Example 7:

```
let numbers = [1, 2, 3];
console.log(numbers.pop()); // Outputs: 3
console.log(numbers); // Outputs: [1, 2]
```

3. **shift():-**Removes and returns the first element, shifting others down.

Example 8:

```
let fruits = ["Apple", "Banana", "Orange"];
console.log(fruits.shift()); // Outputs: Apple
console.log(fruits); // Outputs: ["Banana", "Orange"]
```

4. **unshift():-**Adds elements to the beginning and returns the new length.

Example 9:

```
let fruits = ["Banana", "Orange"];
fruits.unshift("Apple", "Mango");
```

```
console.log(fruits); // Outputs: ["Apple", "Mango", "Banana", "Orange"]
console.log(fruits.unshift("Kiwi")); // Outputs: 5
```

5. **splice()**
    - Adds/removes elements at a specific index.
    - **Syntax**: `splice(start, deleteCount, item1, item2, ...)`

Example 10:

```
 let colors = ["Red", "Green", "Blue"];
colors.splice(1, 1, "Yellow");
console.log(colors); // Outputs: ["Red", "Yellow", "Blue"]
```

6. **reverse()**
    - Reverses the order of elements.

Example 11:

```
 let numbers = [1, 2, 3];
numbers.reverse();
console.log(numbers); // Outputs: [3, 2, 1]
```

7. **sort()**
    - Sorts elements in place (lexicographical by default).

Example 12:

```
 let fruits = ["Banana", "Apple", "Orange"];
fruits.sort();
console.log(fruits); // Outputs: ["Apple", "Banana", "Orange"]
let numbers = [10, 2, 5];
numbers.sort((a, b) => a - b);
console.log(numbers); // Outputs: [2, 5, 10]
```

8. **fill() :-**
    - Fills elements with a static value.
    - **Syntax**: `fill(value, start, end)`

Example 13:

```
 let arr = [1, 2, 3, 4];
arr.fill(0, 1, 3);
console.log(arr); // Outputs: [1, 0, 0, 4]
```

9. **copyWithin()**
    - Copies elements within the array.
    - **Syntax**: `copyWithin(target, start, end)`

Example 14:

```
 let arr = [1, 2, 3, 4, 5];
arr.copyWithin(0, 3, 5);
console.log(arr); // Outputs: [4, 5, 3, 4, 5]
```


-

2.3.2 Accessor Methods

These methods return a new value/array without modifying the original.

1. **concat():-** Merges arrays, returning a new array.

Example 15:

```
 let arr1 = [1, 2];
let arr2 = [3, 4];
let result = arr1.concat(arr2);
console.log(result); // Outputs: [1, 2, 3, 4]
console.log(arr1); // Outputs: [1, 2]
```

2. **slice():-**

   Returns a shallow copy of a portion.

   Syntax: `slice(start, end)`

Example 16:

```
 let fruits = ["Apple", "Banana", "Orange", "Mango"];
let sliced = fruits.slice(1, 3);
console.log(sliced); // Outputs: ["Banana", "Orange"]
```

3. **join():-** Joins elements into a string with a separator.

Example 17:

```
 let fruits = ["Apple", "Banana", "Orange"];
console.log(fruits.join("-")); // Outputs: Apple-Banana-Orange
```

4. **toString() :-** Converts the array to a comma-separated string.

Example 18:

```
 let fruits = ["Apple", "Banana", "Orange"];
console.log(fruits.toString()); // Outputs: Apple,Banana,Orange
```

5. **indexOf():-** Returns the first index of an element or -1 if not found.

Example 19:

```
 let fruits = ["Apple", "Banana", "Orange"];
console.log(fruits.indexOf("Banana")); // 1
console.log(fruits.indexOf("Grape")); // -1
```

6. **lastIndexOf():-** Returns the last index of an element or -1.

Example 20:

```
let numbers = [1, 2, 3, 2, 1];
console.log(numbers.lastIndexOf(2)); // 3
```

7. **includes():-**Checks if an element exists, returning `true`/`false`.

Example 21:

```
let numbers = [1, 2, 3];
console.log(numbers.includes(2)); // true
console.log(numbers.includes(4)); // false
```

8. **toLocaleString():-**Converts elements to a localized string.

Example 22:

```
let prices = [10.99, 25.49];
console.log(prices.toLocaleString("en-US", { style: "currency", currency: "USD" }));
// Outputs: $10.99,$25.49
```

2.3.3 Iteration Methods

These methods execute a callback for each element.

1. **forEach():-**Executes a callback for each element.

Example 23:

```
let fruits = ["Apple", "Banana", "Orange"];
fruits.forEach((item, index) => {
    console.log(`${index}: ${item}`);
});
// Outputs: 0: Apple, 1: Banana, 2: Orange
```

2. **map():-**Creates a new array with mapped values.

Example 24:

```
 let numbers = [1, 2, 3];
let doubled = numbers.map(num => num * 2);
console.log(doubled); // Outputs: [2, 4, 6]
console.log(numbers); // Outputs: [1, 2, 3]
```

3. **filter():-**Creates a new array with elements that pass a test.

Example 25:

```
 let numbers = [1, 2, 3, 4, 5];
let evens = numbers.filter(num => Hiring? Let us help. num % 2 === 0);
console.log(evens); // Outputs: [2, 4]
```

4. **find():-** Returns the first element that satisfies a condition.

Example 26:

```
 let numbers = [1, 2, 3, 4];
let firstEven = numbers.find(num => num % 2 === 0);
console.log(firstEven); // 2
```

5. **findIndex():-**

Returns the index of the first element that satisfies a condition.
-eq **Example 27**:

```
 let numbers = [1, 2, 3, 4];
let firstEvenIndex = numbers.findIndex(num => num % 2 === 0);
console.log(firstEvenIndex); // 1
```

6. **findLast()** (ES2022): -Returns the last element that satisfies a condition.

Example 28:

```
let numbers = [1, 2, 3, 4];
let lastEven = numbers.findLast(num => num % 2 === 0);
console.log(lastEven); // 4
```

7. **findLastIndex()** (ES2022):-Returns the index of the last element that satisfies a condition.

Example 29:

```
let numbers = [1, 2, 3, 4];
let lastEvenIndex = numbers.findLastIndex(num => num % 2 === 0);
console.log(lastEvenIndex); // 3
```

8. **every():-**Checks if all elements satisfy a condition.

Example 30:

```
let numbers = [2, 4, 6];
console.log(numbers.every(num => num % 2 === 0)); // true
```

9. **some():-** Checks if at least one element satisfies a condition.

Example 31:

```
let numbers = [1, 3, 4];
console.log(numbers.some(num => num % 2 === 0)); // true
```

10. **reduce():-** Reduces the array to a single value (left to right).

Example 32:

```
let numbers = [1, 2, 3, 4];
let sum = numbers.reduce((acc, curr) => acc + curr, 0);
console.log(sum); // 10
```

11. **reduceRight():-** Reduces the array to a single value (right to left).

Example 33:

```
 let numbers = [1, 2, 3, 4];
let result = numbers.reduceRight((acc, curr) => acc + curr, 0);
console.log(result); // 10
```

12. **flat():-** Flattens nested arrays up to a specified depth.

Example 34:

```
 let nested = [1, [2, 3], [4, [5]]];
console.log(nested.flat(2)); // Outputs: [1, 2, 3, 4, 5]
```

13. **flatMap():-**Maps elements and flattens the result by one level.

Example 35:

```
 let numbers = [1, 2, 3];
let result = numbers.flatMap(num => [num, num * 2]);
console.log(result); // Outputs: [1, 2, 2, 4, 3, 6]
```

2.3.4 Other Methods

1. **at()** (ES2022) :-Returns the element at a specified index (supports negative indices).

Example 36:

```
 let fruits = ["Apple", "Banana", "Orange"];
console.log(fruits.at(-1)); // Outputs: Orange
```

2. **entries():-**Returns an iterator of array entries (index, value pairs).

Example 37:

```
 let fruits = ["Apple", "Banana"];
for (let [index, value] of fruits.entries()) {
    console.log(`${index}: ${value}`);
}
// Outputs: 0: Apple, 1: Banana
```

3. **keys():-**Returns an iterator of array indices.

Example 38:

```
 let fruits = ["Apple", "Banana"];
for (let key of fruits.keys()) {
    console.log(key);
}
// Outputs: 0, 1
```

4. **values():-** Returns an iterator of array values.

Example 39:

```
 let fruits = ["Apple", "Banana"];
for (let value of fruits.values()) {
    console.log(value);
}
// Outputs: Apple, Banana
```

2.4 Array Prototypes

- **Description**: `Array.prototype` provides methods inherited by all arrays. Custom methods can be added, but this is not recommended.

Example 40:

```
 Array.prototype.customMethod = function() {
    return this.map(item => item.toUpperCase());
};
```

```
let fruits = ["apple", "banana"];
console.log(fruits.customMethod()); // Outputs: ["APPLE", "BANANA"]
```

2.5 Looping Over Arrays

Arrays can be iterated using various methods.

1. **Using for Loop :-**

Example 41:

```
 let animals = ["Dog", "Cat", "Bird"];
for (let i = 0; i < animals.length; i++) {
    console.log(animals[i]);
}
// Outputs: Dog, Cat, Bird
```

2. **Using for...of :-**

Example 42:

```
 let animals = ["Dog", "Cat", "Bird"];
for (let animal of animals) {
    console.log(animal);
}
// Outputs: Dog, Cat, Bird
```

3. **Using for...in** (Not Recommended):-Iterates over enumerable properties, may include prototype properties.

Example 43:

```
 let animals = ["Dog", "Cat", "Bird"];
for (let index in animals) {
    console.log(animals[index]);
}
// Outputs: Dog, Cat, Bird
```

2.6 Practical Example: Combining Loops, Arrays, and Functions

Example 44:

```
function processArray(arr) {
    let evens = arr.filter(num => num % 2 === 0);
    let doubled = evens.map(num => num * 2);
    let sum = doubled.reduce((acc, curr) => acc + curr, 0);
    return sum;
}
let numbers = [1, 2, 3, 4, 5, 6];
console.log(processArray(numbers)); // Outputs: 24
```

---

3. Functions in JavaScript

3.1 Function Definition

A function is a reusable block of code designed to perform a specific task, defined using the `function` keyword.

Example 45:

```
function greet() {
    console.log("Hello, World!");
}
greet(); // Outputs: Hello, World!
```

3.2 Types of Functions

1. **Named Functions:-** Functions with a specific name.

Example 46:

```
function sayHello() {
    console.log("Hello!");
}
sayHello(); // Outputs: Hello!
```

2. **Anonymous Functions:-** Functions without a name, often used as arguments.

Example 47:

```
 setTimeout(function() {
    console.log("Delayed!");
}, 1000);
// Outputs: Delayed! (after 1 second)
```

3. **Arrow Functions :-** Concise ES6 syntax.

Example 48:

```
 const add = (a, b) => a + b;
console.log(add(2, 3)); // Outputs: 5
```

4. **Immediately Invoked Function Expressions (IIFE):-** Functions that run immediately after definition.

Example 49:

```
 (function() {
    console.log("IIFE executed!");
})();
// Outputs: IIFE executed!
```

3.3 Pre/User-Defined Functions

1. **Pre-defined Functions :-** Built-in JavaScript functions like `parseInt()`, `alert()`.

Example 50:

```
 console.log(parseInt("123")); // Outputs: 123
```

2. **User-Defined Functions :-** Custom functions created by the programmer.

Example 51:

```
 function multiply(a, b) {
    return a * b;
}
console.log(multiply(4, 5)); // Outputs: 20
```

3.4 Arguments and Returning Values

Functions can take arguments and return values using `return`.

Example 52:

```
 function calculateArea(length, width) {
    return length * width;
}
console.log(calculateArea(5, 3)); // Outputs: 15
```